

Ciallo: A Structured Search-Based Framework for Automated Program Repair with Large Language Models

JUNLANG FENG, School of Advanced Manufacturing Guangdong University of Technology, China

FANLONG ZHANG*, School of Computer Science Guangdong University of Technology, China

HUAXIN WEI, School of Advanced Manufacturing Guangdong University of Technology, China

ZIPING WANG, Guangzhou Institute of Educational Research, China

JIANQI LIU, School of Computer Science Guangdong University of Technology, China

Recent advances in large language models (LLMs) have significantly improved automated program repair (APR), particularly through conversational frameworks that iteratively refine patches using execution feedback. However, existing approaches treat repair as an unstructured conversational process, implicitly conflating defect understanding and patch generation, which leads to inefficient and unstable exploration in a large patch space. To address this limitation, we reformulate LLM-based program repair as a structured search problem and introduce the *understanding-repair paradigm*, which explicitly separates defect comprehension from repair execution. Based on this paradigm, we propose CIALLO, a structured search-based repair framework that decomposes repair into semantic grounding, hypothesis generation, instantiation, pruning, and expansion. The framework leverages prompt chaining to enable structured reasoning across stages and introduces a novel *Patch Sieve* mechanism to prune unpromising candidates based on failure-set dominance. Experimental results on Defects4J and HumanEval-Java show that CIALLO outperforms state-of-the-art APR frameworks by 38.3% and 15.0%, respectively, while significantly reducing redundant patch exploration. These results demonstrate that modeling repair as structured search enables more effective, efficient, and controllable LLM-based program repair.

CCS Concepts: • **Software and its engineering** → **Maintaining software**; **Software testing and debugging**.

Additional Key Words and Phrases: Automated program repair, large language models, prompt chaining, understanding-repair paradigm, conversational frameworks, prompt engineering.

ACM Reference Format:

Junlang Feng, Fanlong Zhang, Huaxin Wei, Ziping Wang, and Jianqi Liu. 2018. Ciallo: A Structured Search-Based Framework for Automated Program Repair with Large Language Models. *J. ACM* 37, 4, Article 111 (August 2018), 28 pages. <https://doi.org/XXXXXXX.XXXXXXX>

*Fanlong Zhang is the corresponding author.

Authors' Contact Information: Junlang Feng, School of Advanced Manufacturing and Guangdong University of Technology, Guangzhou, Guangdong, China; Fanlong Zhang, zhangfanlong@gdut.edu.cn, School of Computer Science and Guangdong University of Technology, Guangzhou, Guangdong, China; Huaxin Wei, School of Advanced Manufacturing and Guangdong University of Technology, Guangzhou, Guangdong, China; Ziping Wang, Guangzhou Institute of Educational Research, Guangzhou, Guangdong, China; Jianqi Liu, liujianqi@gdut.edu.cn, School of Computer Science and Guangdong University of Technology, Guangzhou, Guangdong, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-735X/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Automated program repair (APR) aims to automatically generate correct patches for defective programs while preserving intended behavior [15, 18, 23]. Over the past decade, a wide range of APR techniques have been proposed, including search-based, learning-based, and hybrid approaches [2, 8, 16, 33, 35, 41–43]. Despite these advances, repairing complex real-world defects remains challenging due to the vastness of the patch space and the difficulty of accurately understanding defect causes.

Recent advances in large language models (LLMs) have significantly reshaped the landscape of APR. By leveraging strong code understanding and generation capabilities, LLM-based repair frameworks have demonstrated remarkable effectiveness in fixing defects that are difficult for traditional approaches [2, 8, 16, 35, 43]. Most existing frameworks adopt a *conversational-style repair paradigm*, in which models iteratively refine candidate patches through interaction with execution feedback and prompt-level guidance [35]. In this paradigm, the LLM is treated as a black-box reasoning engine that incrementally improves candidate solutions through multi-turn interactions.

While effective in practice, this paradigm suffers from a fundamental limitation: it treats program repair as an *unstructured conversational refinement process*. In particular, defect understanding and patch generation are implicitly intertwined within a single interaction loop, requiring the model to simultaneously infer defect causes, explore repair strategies, and generate patches. As a result, the repair process relies heavily on implicit reasoning and extensive trial-and-error in a large patch space, often leading to inefficient and unstable exploration. Fundamentally, existing approaches lack an explicit structure for modeling repair as a search process, which limits their ability to systematically explore and control the patch space.

While effective in practice, existing conversational APR frameworks suffer from a fundamental limitation: they treat program repair as an *unstructured conversational refinement process*. In these frameworks, defect understanding and patch generation are implicitly intertwined within a single interaction loop, requiring the model to simultaneously infer defect causes, explore repair strategies, and generate candidate patches. As a result, the repair process relies heavily on implicit reasoning and extensive trial-and-error in a large patch space, often leading to inefficient exploration and unstable repair behavior.

This limitation becomes more pronounced as the repair process deepens. Due to context window constraints and the absence of explicit reasoning states, LLMs gradually lose track of previously explored repair hypotheses, resulting in redundant exploration and loss of search consistency. Long conversational histories further exacerbate this issue, causing models to revisit previously explored regions or prematurely reset the search process. Consequently, although conversational APR frameworks can succeed in individual cases, their repair process remains difficult to control, inefficient at scale, and highly sensitive to prompt ordering and context length.

These observations suggest that a core challenge in LLM-based program repair lies in enabling models to explicitly reason about defect causes and systematically explore the patch space. To address this challenge, we argue that effective program repair should be formulated as a *structured search problem*, rather than an unstructured conversational process. Based on this insight, we propose the *understanding–repair paradigm*, which explicitly separates defect understanding from repair execution. Under this paradigm, LLMs are first guided to perform *semantic grounding*, producing structured intermediate reasoning states that capture defect semantics and repair hypotheses. Repair is then conducted as a controlled exploration over these hypotheses, instead of relying on unconstrained conversational refinement. Fig. 1 illustrates the key difference between the conventional conversational paradigm and the proposed understanding–repair paradigm. In contrast to existing approaches that rely on implicit trial-and-error, our paradigm explicitly models

defect understanding and repair as separate yet coordinated stages, thereby enabling more efficient and controllable exploration of the patch space.

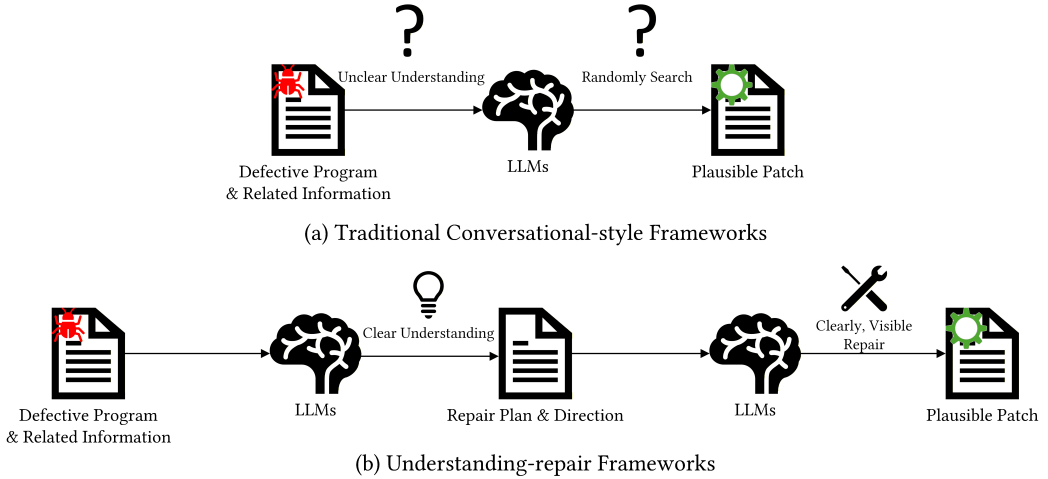


Fig. 1. Comparison between the conversational-style repair paradigm and the proposed understanding-repair paradigm. The conversational paradigm treats repair as an unstructured iterative refinement process, where defect understanding and patch generation are implicitly intertwined. In contrast, the understanding-repair paradigm explicitly separates semantic grounding from repair execution, enabling structured exploration of repair hypotheses and more efficient navigation of the patch space.

Building upon this paradigm, we present CIALLO, a structured search-based framework for automated program repair. CIALLO instantiates the understanding-repair paradigm by decomposing repair into a sequence of coordinated stages, including semantic grounding, hypothesis generation, instantiation, pruning, and expansion. At the system level, CIALLO leverages *prompt chaining* to implement structured reasoning across stages, enabling explicit construction and navigation of repair hypotheses. This design transforms repair from a single-trajectory conversational process into a controlled exploration over multiple candidate paths. At the algorithmic level, CIALLO introduces a novel *Patch Sieve* mechanism, which acts as a dominance-based pruning operator that removes unpromising candidates based on test failure outcomes. This mechanism effectively reduces redundant exploration and improves the efficiency of patch search. As demonstrated in our evaluation, this structured design enables both more effective and more efficient repair compared to existing conversational APR frameworks.

In summary, this paper makes the following contributions:

- (1) We introduce the *understanding-repair paradigm* for LLM-based automated program repair, which reformulates repair as a structured search problem by explicitly separating defect understanding from repair execution.
- (2) We propose CIALLO, a structured search-based repair framework that instantiates this paradigm by integrating semantic grounding, hypothesis generation, and iterative pruning into a unified process, while leveraging prompt chaining to enable structured reasoning across stages.
- (3) We design *Patch Sieve*, a dominance-based pruning mechanism that reduces redundant exploration and improves search efficiency.

- (4) We conduct a comprehensive evaluation on Defects4J and HumanEval-Java, demonstrating that CIALLO consistently outperforms state-of-the-art approaches in both effectiveness and efficiency.

The remainder of this paper is organized as follows. Section 2 reviews related work on automated program repair and LLM-based repair frameworks. Section 3 presents the understanding–repair paradigm and the CIALLO framework. Section 4 describes the experimental setup and evaluation results. Section 5 discusses threats to validity. Section 6 concludes the paper.

2 RELATED WORK

2.1 Classical and Early LLM-based APR Techniques

Prior surveys categorize automated program repair (APR) techniques into several major paradigms, including search-based, pattern-based, semantic-based, and learning-based approaches [19, 23, 37]. Search-based methods [18, 23] formulate repair as an optimization problem over a predefined patch space guided by test execution, while pattern-based approaches, such as PAR [15], rely on transformation rules derived from human-written patches. Semantic-based techniques, such as SemFix [24], leverage symbolic execution and constraint solving to synthesize patches that satisfy inferred specifications. Learning-based methods [7, 27, 44] further attempt to capture repair patterns through data-driven models trained on historical bug-fixing commits.

Prior work has identified fundamental challenges faced by these approaches when applied to complex real-world defects, primarily due to the combinatorial explosion of the patch space and the difficulty of modeling defect semantics [19, 23]. As a result, their effectiveness is often limited to relatively well-defined bug types.

Early research on learning-based APR explored neural models for learning repair patterns from large-scale datasets. SequenceR [3] formulates program repair as a sequence-to-sequence translation task with a copy mechanism, while DLFix [20] incorporates tree-based neural representations to model both code context and transformation patterns. Subsequent studies investigate the use of pre-trained code models, including fine-tuning CodeBERT for program repair [22] and template-based generation methods such as Recoder [45]. These approaches demonstrate that neural models can learn meaningful repair patterns without relying on manually designed templates. However, existing studies also indicate that such models are typically trained on specific datasets or bug categories, and their generalization to unseen defects remains limited [7].

With the emergence of large language models (LLMs), early LLM-based APR approaches have been categorized into fine-tuning-based, in-context learning-based, and cloze-style paradigms [1, 37]. Fine-tuning-based methods adapt LLMs to repair tasks using task-specific datasets [6, 7, 27, 44], while in-context learning-based approaches leverage zero- or few-shot prompting without parameter updates [12, 33]. Cloze-style methods further assume known defect locations and formulate repair as masked code completion [25, 31, 33].

Overall, both classical and early LLM-based APR approaches primarily focus on improving patch generation accuracy at the syntactic or local level. Prior work also observes that these methods largely treat repair as a single-step or locally guided transformation process, with limited explicit modeling of global program semantics or systematic reasoning about defect causes [32, 39]. These limitations have motivated the development of conversational LLM-based repair frameworks, which aim to address these challenges through iterative interaction and feedback.

2.2 LLM-based Conversational Program Repair

In response to the limitations of single-step repair paradigms, recent work has explored conversational frameworks that leverage the interactive capabilities of large language models (LLMs) to

enable iterative, feedback-driven program repair, aiming to overcome the opacity and rigidity of traditional generate-and-validate pipelines [32, 39]. The pioneering work in this direction is ChatRepair, proposed by Xia and Zhang [32, 34], which demonstrates that conversational interaction with LLMs can effectively repair a substantial number of real-world bugs from the Defects4J benchmark [14] at a modest cost. ChatRepair establishes the foundational conversational paradigm, in which the LLM takes test failure information as input, generates candidate patches, and iteratively refines them through multi-turn dialogue guided by feedback from previous attempts.

Subsequent work extends this paradigm along several dimensions. ContrastRepair [16] augments repair with contrastive test cases, CigaR [8] improves efficiency via patch reuse and prompt compression, D4C [36] aligns prompt design with LLM training objectives, and ReduceFix [38] reduces failure-inducing inputs prior to generation. Prior studies observe that these methods collectively illustrate the maturation of the conversational paradigm while preserving its core iterative refinement mechanism [32].

More recently, agent-based approaches have further extended the conversational paradigm by introducing structured workflows and autonomous decision-making. RepairAgent [2] adopts a state-machine architecture to coordinate specialized agents, while AdverIntent-Agent [43] leverages multi-agent adversarial reasoning. Prior work notes that, despite architectural differences, both conversational and agent-based methods share a common design principle: they treat program repair as an iterative, feedback-driven refinement process, and recent surveys categorize them together under the umbrella of dialogue-based APR [1, 21, 32, 37, 39, 42].

However, existing studies further indicate that this design introduces several fundamental limitations that stem from the lack of explicit structural grounding in the repair process. First, the absence of explicit defect understanding forces LLMs to explore the patch space in a trial-and-error manner, leading to inefficient and unstable search [10, 39]. Second, the reliance on coarse-grained feedback (e.g., test pass/fail outcomes) provides limited guidance for reasoning about defect causes, making it difficult to systematically refine repair hypotheses [10]. Third, as conversation depth increases, the lack of explicit reasoning states hinders the model's ability to track and reuse prior repair attempts, resulting in redundant exploration [32].

Recent work has attempted to mitigate these issues by incorporating more fine-grained feedback signals, such as token-level uncertainty [42] and execution-level dynamic information [10]. Agent-based approaches partially address these limitations by introducing dedicated reasoning stages and state transitions [2, 39, 43]. Nevertheless, prior studies note that even in these frameworks, the reasoning process remains largely driven by autonomous model decisions expressed in free-form natural language, without explicit and controllable mechanisms for structuring defect understanding in relation to program syntax and semantics [1, 37].

These observations collectively highlight a critical research gap, namely the need for repair frameworks that retain the advantages of iterative, feedback-driven dialogue while introducing explicit structural discipline into defect understanding and repair reasoning. In the following section, we examine how structured reasoning techniques developed in the broader code generation domain can provide a principled foundation for addressing this challenge.

2.3 Structuring Reasoning in LLM-based Repair

Recent advances in large language models have highlighted the importance of structured reasoning for solving complex tasks. Chain-of-Thought (CoT) prompting [28] demonstrates that exposing intermediate reasoning steps can improve model performance, and subsequent work has explored extensions such as Tree-of-Thought (ToT) [40] and Graph-of-Thought (GoT) [40], which enable more systematic exploration of reasoning trajectories. In parallel, prompt chaining techniques have been developed to decompose complex tasks into sequences of structured subtasks with explicit

intermediate outputs [29, 30]. These approaches have been widely applied in code generation, where multi-stage reasoning frameworks guide models through intermediate steps such as requirement analysis and implementation [9, 17]. Prior work suggests that structured decomposition of reasoning processes can improve the reliability and consistency of LLM-generated outputs in complex generation tasks [26].

In contrast, the adoption of structured reasoning in automated program repair remains limited. As discussed in Section 2.2, conversational and agent-based APR frameworks primarily rely on multi-turn interactions with LLMs, which can be viewed as a coarse-grained form of iterative reasoning. Some approaches incorporate CoT-style prompts to elicit intermediate explanations [32, 42]. However, existing studies observe that these methods treat structured reasoning primarily as a prompt-level enhancement rather than as a core design principle. Defect understanding and patch generation remain tightly coupled, and intermediate reasoning states are not explicitly represented or systematically reused across repair iterations [1, 37]. As a result, the repair process lacks the structural discipline observed in code generation and related tasks [10].

These observations collectively indicate a gap between advances in structured reasoning and their application to program repair. While prior work suggests that iterative dialogue improves flexibility, the absence of explicit structural grounding limits the controllability and efficiency of repair processes [10, 32, 39]. In particular, existing approaches lack mechanisms for explicitly representing, reusing, and verifying defect understanding across repair iterations, which has been shown to contribute to redundant exploration and unstable repair behavior. This gap motivates further investigation into how structured reasoning can be systematically incorporated into LLM-based repair frameworks.

3 METHOD

3.1 Overview

CIALLO is a structured search-based framework for automated program repair, which reformulates LLM-based repair as a controlled search process over the patch space. Unlike conventional conversational repair frameworks, which rely on implicit reasoning within iterative interactions, CIALLO explicitly separates defect understanding from repair execution and organizes the repair process into structured reasoning stages. From a high-level perspective, CIALLO formulates program repair as a search problem over a space of repair hypotheses. Each hypothesis corresponds to a potential explanation of the defect and its associated repair strategy. The goal is to efficiently explore this hypothesis space to identify semantically correct patches.

To achieve this, CIALLO organizes the repair process into four phases: *Defect Understanding*, *Direction Generation*, *Patch Generation and Validation*, and *Patch Multiplication*. These phases can be interpreted as stages of structured search, including semantic grounding, hypothesis generation, instantiation, pruning, and expansion.

In the *Defect Understanding* phase, the large language model (LLM) analyzes the defect using a sequence of chained prompts, each representing an explicit reasoning state. This phase aims to identify the root cause of the defect and construct a structured semantic understanding, thereby reducing the uncertainty of the repair space. From a search perspective, this phase corresponds to semantic grounding, which reduces uncertainty in the patch space.

Based on the inferred defect understanding, CIALLO enters the *Direction Generation* phase, where multiple repair directions are proposed. Each direction corresponds to a distinct repair hypothesis, enabling structured and diversified exploration of the patch space, in contrast to single-trajectory conversational repair. This phase enables structured exploration by introducing multiple repair hypotheses.

In the *Patch Generation and Validation* phase, CIALLO generates candidate patches for each repair direction and validates them against the test suite. From a search perspective, this phase corresponds to hypothesis instantiation. Intermediate patches are filtered using the proposed *Patch Sieve* mechanism, which removes dominated patches according to test failure outcomes. This phase instantiates repair hypotheses into executable patches, bridging high-level reasoning and concrete program modification. The Patch Sieve mechanism acts as a pruning operator, eliminating candidates that do not reduce the failure set.

Once a plausible patch is obtained, CIALLO enters the *Patch Multiplication* phase. Instead of returning a single plausible patch, the framework generates additional patches guided by the existing plausible solution, thereby expanding promising regions of the search space and improving robustness against overfitting to limited test cases. This phase performs search expansion around promising candidates, improving robustness against overfitting.

Overall, CIALLO can be viewed as a structured search framework that systematically explores, prunes, and refines the patch space, enabling efficient and controllable program repair.

Fig. 2 illustrates the overall architecture of CIALLO, while Algorithm 1 summarizes the structured search process, highlighting the interaction between hypothesis generation, instantiation, pruning, and expansion. The detailed implementation of the *ExecutePatchGeneration* procedure is provided in Appendix B.

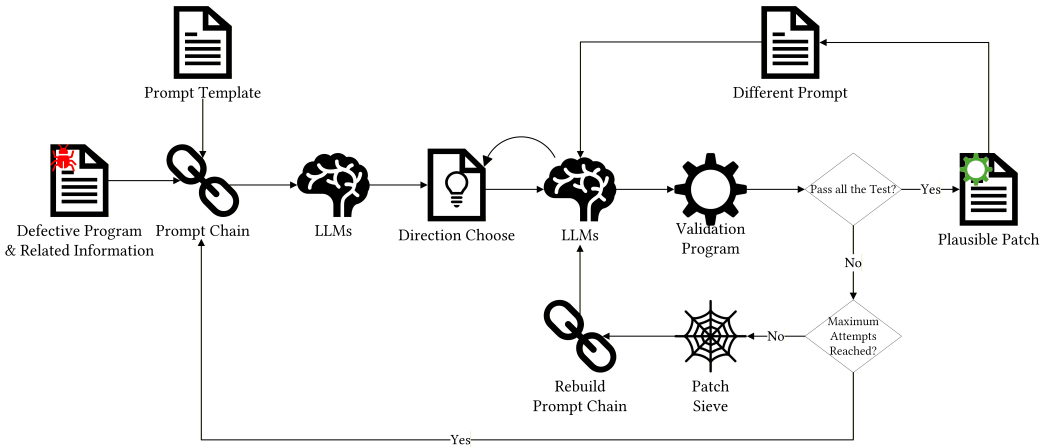


Fig. 2. CIALLO: Structured search process for automated program repair

3.2 Understanding–Repair Paradigm

Existing LLM-based program repair frameworks primarily adopt a conversational paradigm, in which defect understanding and patch generation are implicitly intertwined within iterative interactions. While effective in certain scenarios, this paradigm relies heavily on implicit reasoning, resulting in inefficient and unstable exploration of the patch space.

In this work, we propose the *understanding–repair paradigm*, which explicitly separates defect understanding from repair execution. Instead of directly generating patches, the framework first constructs explicit representations of defect semantics, which are then used to guide subsequent repair.

Formally, the paradigm decomposes program repair into two stages: (i) *understanding*, which models the semantic causes of defects, and (ii) *repair*, which explores the patch space conditioned

Algorithm 1 CIALLO Repair Framework

Input: Defective program δ , test suite τ , repair modes $\mathcal{M}$, maximum rounds R , max prompt length for initial generation L_f , max prompt length for multiplication L_m , target number of plausible patches N

Output: Set of plausible patches Π

```

1: procedure CIALLO( $\delta, \tau, \mathcal{M}, R, L_f, L_m, N$ )
2:    $r \leftarrow 0$ 
3:    $\Pi \leftarrow \emptyset$ 
4:    $\phi \leftarrow \text{CollectInfo}(\delta, \tau)$ 
5:   while  $r < R$  do
6:      $\Pi_{\text{iter}} \leftarrow \emptyset$ 
7:     for each mode  $m \in \mathcal{M}$  do
8:       if not PlausiblePatchesExist( $\Pi_{\text{iter}}$ ) then
9:          $\alpha \leftarrow \text{UnderstandDefect}(\delta, m)$ 
10:         $\Gamma \leftarrow \text{GenerateDirections}(\delta, m, \alpha, \phi)$ 
11:         $\Pi_{\text{iter}} \leftarrow \text{GenerateInitialPatches}(\delta, m, \Gamma, \tau, L_f)$ 
12:         $\Pi_{\text{iter}} \leftarrow \text{PatchSieve}(\Pi_{\text{iter}}, \tau)$ 
13:       else
14:          $\Pi_{\text{iter}} \leftarrow \text{PatchMultiplication}(\delta, m, \Pi_{\text{iter}}, L_m, N)$ 
15:      $\Pi \leftarrow \text{GetPlausiblePatches}(\Pi_{\text{iter}})$ 
16:     if PlausiblePatchesExist( $\Pi$ ) then break
17:      $r \leftarrow r + 1$ 
18: return  $\Pi$ 

```

on the constructed understanding. This separation enables more structured reasoning, reduces reliance on implicit inference, and improves the controllability of the repair process.

From a search perspective, the understanding stage corresponds to constructing repair hypotheses, while the repair stage corresponds to exploring and instantiating these hypotheses. By explicitly modeling this two-stage process, the proposed paradigm transforms program repair from an unstructured conversational refinement task into a structured and guided search problem. This perspective provides a principled foundation for designing structured search-based repair frameworks.

3.3 Defect Understanding

Accurate identification of defect semantics is a prerequisite for effective automated program repair. However, existing conversational-style repair frameworks typically entangle defect understanding with patch generation, forcing LLMs to implicitly infer defect causes during code synthesis. This design often leads to unstable reasoning and inefficient exploration of the patch space.

To address this limitation, CIALLO explicitly introduces a *Defect Understanding* phase, which decouples defect analysis from repair execution. This phase operationalizes the understanding–repair paradigm by transforming implicit reasoning into an explicit semantic modeling process. Specifically, the goal of this phase is to construct a structured representation of defect semantics that serves as the foundation for subsequent repair.

To enable structured reasoning, we adopt the *prompt chaining* technique, which decomposes defect understanding into a sequence of intermediate reasoning steps. Each step corresponds to a specific reasoning state, guiding the LLM to progressively analyze the defect, identify potential root causes, and summarize repair-relevant information. Compared to single-prompt approaches, this design reduces reasoning ambiguity and improves the stability of defect interpretation.

Since different repair scenarios require different contextual information, the prompt chain is dynamically configured based on defect characteristics and available diagnostic signals. The prompt templates used in this phase are illustrated in the green blocks of Fig. 3(a) and (b). Rather than using fixed prompts, CIALLO adapts prompt composition at runtime, providing only task-relevant information for each reasoning step. This dynamic construction reduces irrelevant context and improves the efficiency and focus of reasoning.

In particular, when the defect triggers a runtime exception, CIALLO extracts an *error trajectory* from the compiler or runtime stack trace. The line where the exception occurs is first identified, and a code fragment centered at the error line is selected as additional input. If multiple stack frames correspond to the current file, the extracted fragments are ordered according to the stack trace. To avoid excessive context caused by deep recursion or repeated calls, the number of fragments is limited to at most twenty.

Fig. 3 illustrates the prompt structure used in this phase. Different colors represent different prompt sources, including system prompts, template-generated prompts, and LLM responses. The bold text indicates scenario-dependent content, and bold italic text denotes components that appear only in specific repair situations.

Overall, this phase reduces the uncertainty of the repair space by explicitly modeling defect semantics, thereby enabling more accurate and guided exploration in subsequent repair stages.

3.4 Direction Generation

During preliminary experiments, we observed that LLM-based repair often produces patches that are highly similar to previously generated ones, leading to redundant exploration and reduced search efficiency. This limitation arises from the inherent tendency of conversational repair to follow a single reasoning trajectory, which restricts the diversity of explored solutions.

To address this issue, CIALLO introduces a *Direction Generation* phase, in which the LLM first proposes multiple high-level repair directions before generating concrete patches. Each direction represents a distinct repair hypothesis, allowing the framework to explore the patch space in a structured and diversified manner. This design transforms repair from single-path reasoning into controlled hypothesis exploration.

From a search perspective, this phase corresponds to hypothesis exploration, where alternative repair strategies are generated based on the defect understanding constructed in the previous phase. By explicitly enumerating multiple repair directions, CIALLO avoids repeatedly exploring similar solutions and significantly improves the coverage of the search space.

The prompt used for direction generation is illustrated in the blue blocks of Fig. 3(b). Before generating directions, the prompt chain provides the LLM with contextual information and defect analysis results obtained in the defect understanding phase. Compared with the prompts used in earlier stages, the prompt here is simplified to retain only the information necessary for direction reasoning. This design reduces token consumption and allows the model to focus on generating repair strategies, demonstrating the flexibility enabled by prompt chaining.

Instead of explicitly specifying the number of directions, CIALLO allows the LLM to determine a context-appropriate number of candidate strategies. In practice, the model typically produces between five and ten directions, which provides sufficient diversity while avoiding excessive exploration cost.

Overall, this phase improves search diversity by explicitly modeling alternative repair hypotheses, thereby enabling more effective and comprehensive exploration of the repair space.

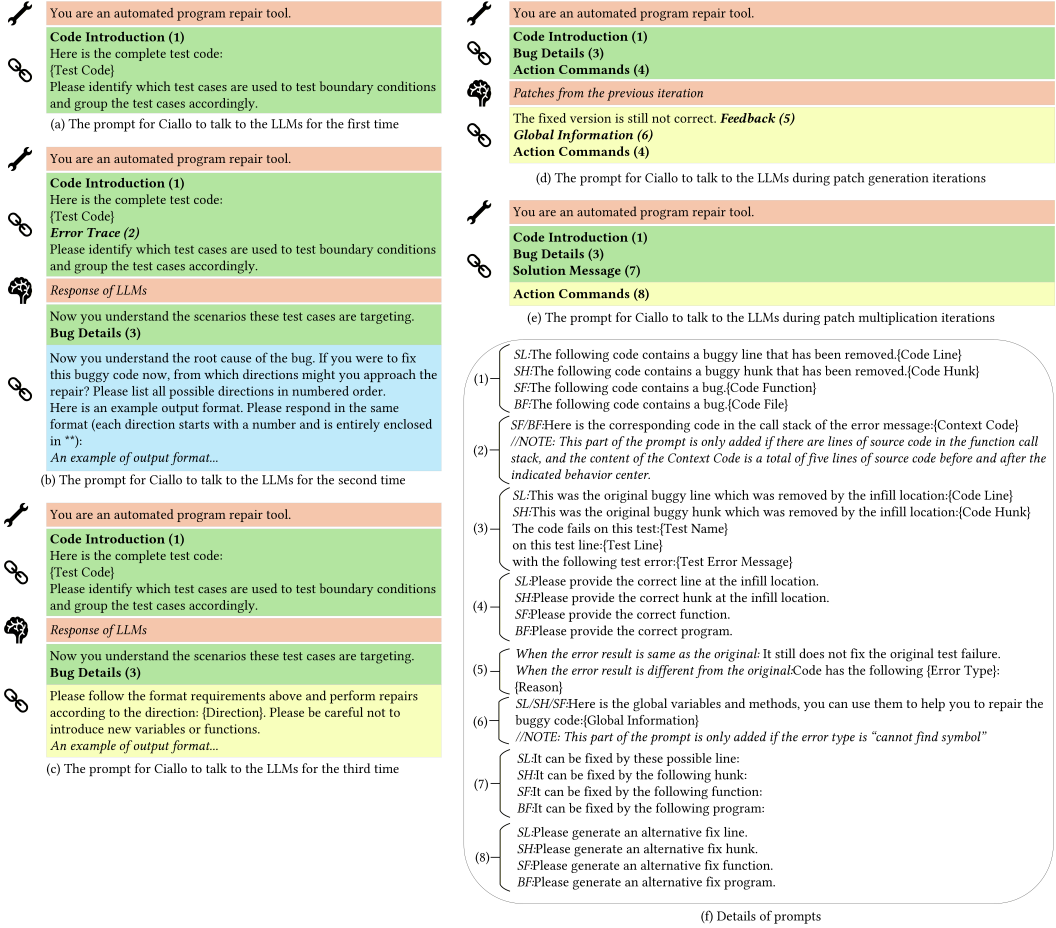


Fig. 3. Prompt structure used in CIALLO. The figure illustrates the prompt chaining process for defect understanding under different repair scenarios. Different colors denote different prompt sources, including system prompts, template-generated prompts, and LLM responses. Bold text indicates scenario-dependent content, while bold italic text denotes optional components that appear only in specific repair situations.

3.5 Patch Generation and Validation

In this phase, CIALLO generates candidate patches for each repair direction produced in the previous phase. Instead of combining all directions in a single prompt, each direction is instantiated independently, as illustrated in Fig. 3(c). This design ensures that different repair hypotheses are explored without mutual interference, thereby improving the coverage of the search space.

From a search perspective, this phase corresponds to hypothesis instantiation, where high-level repair strategies are translated into concrete program modifications. By isolating each direction, CIALLO avoids cross-contamination between hypotheses and enables more controlled exploration.

Unlike conventional repair pipelines, CIALLO does not validate patches immediately after generation. Instead, all candidate patches within an iteration are generated first, followed by a validation stage. This design serves two purposes. First, since passing the validation test suite does not guarantee correctness, generating multiple plausible patches increases the likelihood of finding a

correct fix. Second, because repair directions are independent, patch generation and validation can be parallelized, which provides potential efficiency gains (although our current implementation executes these steps sequentially for simplicity).

After generation, each patch is evaluated using a trusted test suite, and the results are categorized into three types: (1) passed, (2) compilation error, and (3) failed test. If any patch passes all test cases, CIALLO proceeds to the next phase. Otherwise, partially correct patches are further processed using the proposed *Patch Sieve* mechanism.

Patch Sieve serves as a pruning strategy to retain promising patches while eliminating unproductive ones. Let I denote the set of failing tests for the original defective program, and let G denote the set of failing tests for a generated patch. A patch is accepted if the following condition holds:

$$PS \equiv (I \cap G) \subset I \quad (1)$$

This condition ensures that a patch is preserved only if it reduces the original failure set, while patches that introduce new failures without fixing existing ones are discarded. From a search perspective, Patch Sieve implements dominance-based pruning, where patches that do not improve the objective (i.e., reducing failing tests) are eliminated from further exploration.

Patch Sieve is not applied in the first iteration. All generated patches are initially retained, allowing the LLM to refine implementations that follow correct repair directions but contain imperfect code. From the second iteration onward, patches that do not satisfy the Patch Sieve condition are removed, which enables early elimination of incorrect directions and improves search efficiency.

Patches that pass Patch Sieve are incorporated into the prompt for the next iteration, as shown in Fig. 3(d). Different prompts are constructed for different failed patches, allowing the LLM to refine repairs based on specific validation feedback. Patches that pass Patch Sieve are incorporated into the prompt for the next iteration, as shown in Fig. 3(d). Different prompts are constructed for different failed patches, allowing the LLM to refine the repair based on the specific validation feedback.

For certain compilation errors, additional global context is introduced. In particular, when unresolved symbol errors occur within a localized function, global variables and function signatures from the current file are added to the prompt, which enhances the retrieval of relevant information.

Overall, this phase bridges high-level repair hypotheses and concrete program modifications, while the Patch Sieve mechanism ensures efficient exploration through principled pruning of the search space.

3.6 Patch Multiplication

Even if a generated patch passes all validation tests, it may still be incorrect, since the available test suite may not fully cover all program behaviors. Therefore, relying on a single plausible patch is insufficient for reliable program repair. To improve robustness, CIALLO introduces a *Patch Multiplication* phase, which generates additional plausible patches that are syntactically or semantically distinct from existing ones. This phase aims to expand promising regions of the search space, thereby increasing the probability of discovering correct patches.

From a search perspective, this phase corresponds to hypothesis expansion, where new candidate patches are generated based on previously identified plausible solutions. Unlike the previous phase, which focuses on instantiating repair hypotheses, Patch Multiplication explores local neighborhoods around promising candidates, enabling more comprehensive coverage of the repair space.

Our design is inspired by CigaR [8], which adopts a beam search strategy. However, beam search is not suitable for our framework. Since CIALLO generates multiple repair directions in earlier stages, restricting exploration to a narrow beam may prematurely discard the correct repair direction.

Instead, we adopt a level-order exploration strategy, which expands patches layer by layer while preserving diversity across different directions. Based on empirical observations, we limit the maximum search depth to three layers. In addition, the search terminates early once the number of plausible patches reaches a predefined threshold (15 in our implementation). The prompt used in this phase is illustrated in Fig. 3(e).

Different from the previous phase, only plausible patches are used to construct prompts, while incorrect patches are discarded. Moreover, patches that have already been used in previous iterations are excluded from immediate reuse, which prevents redundant exploration.

To mitigate premature termination caused by insufficient initial patches, we introduce a reconstruction strategy that selectively reuses previously explored patches. Formally, let τ_n denote the target number of patches, π_p the current set of plausible patches, and π_u the ordered set of previously used patches. If the number of available patches is insufficient, the oldest used patches are removed to allow reuse.

$$k = \tau_n - |\pi_p| \quad (2)$$

where k is the number of patches to be removed from π_u .

The updated used patch set π'_u is defined as

$$\pi'_u = \begin{cases} \{p_{k+1}, \dots, p_m\} & , m > k \\ \emptyset & , m \leq k \end{cases} \quad (3)$$

where $\pi_u = \{p_1, \dots, p_m\}$ and the ordering reflects usage time. The patches available for the next iteration are then given by

$$\pi_p - \pi'_u \quad (4)$$

This reconstruction strategy ensures that the search process maintains diversity while avoiding repeated exploration, thereby improving robustness and stability.

Overall, this phase expands promising regions of the repair space through structured exploration, completing the transition from hypothesis construction to exploration, instantiation, pruning, and expansion within the CIALLO framework.

4 EVALUATION

To comprehensively evaluate the effectiveness of CIALLO, we design experiments to answer the following research questions:

- **RQ1: How does CIALLO compare with state-of-the-art APR frameworks?** We evaluate CIALLO on the widely used Defects4J [14] and HumanEval-Java [13] benchmarks, and compare it with recent LLM-based APR systems to assess overall repair effectiveness.
- **RQ2: How robust is CIALLO across different LLMs?** We evaluate CIALLO using GPT-4o [11] and DeepSeek-V3 [4], which differ in architecture and training data, to study the stability and generality of the proposed framework.
- **RQ3: What types of defects benefit from CIALLO?** Following prior work, we categorize defects into four types (single-line, single-hunk, single-function, and beyond-function), and analyze repair effectiveness across different defect categories.
- **RQ4: How does each component of CIALLO contribute to repair performance?** We conduct ablation studies on the key components, including Defect Understanding, Direction Generation and Patch Sieve, to quantify their individual contributions.

- **RQ5: How efficient is Ciallo in terms of token cost for defect repair?** Following prior work, we evaluate the efficiency of CIALLO in terms of token consumption and monetary cost, and compare it with state-of-the-art frameworks using publicly reported cost data.
- **RQ6: How does CIALLO explore and refine the patch space during repair?** We analyze the search dynamics of CIALLO by examining the generation volume of repair directions, the distribution of computational effort across stages, and the quantitative impact of the Patch Sieve mechanism on search space pruning.

4.1 Experimental Setup

4.1.1 Environment. All experiments were conducted on a machine running Ubuntu 22.04 with an Intel i9-12900K CPU and 64GB RAM. The framework was implemented in Python, and multiple Java runtimes were used to support different benchmarks. Docker containers were employed to ensure reproducibility and cross-platform portability. We evaluate CIALLO using two large language models: GPT-4o and DeepSeek-V3-0324, which differ in architecture and training data, allowing us to study the robustness of the proposed framework.

4.1.2 Hyperparameter. Following prior work [16, 34, 35], we limit the maximum number of repair rounds to 22. In each round, up to three patch generation attempts are allowed. The temperature of the LLM is set to 1 to encourage diverse patch generation. During repair, CIALLO aims to obtain up to 15 plausible patches for each defect. Once the target number is reached, the framework stops further exploration and proceeds to the next defect. These parameters were selected based on prior studies and preliminary experiments.

4.1.3 Benchmark. We evaluate CIALLO on two widely used APR benchmarks: Defects4J [14] and HumanEval-Java [13]. Defects4J contains real-world defects from large-scale open-source Java projects and is commonly used to evaluate program repair systems. We use Defects4J 3.0, which includes 854 defects. To ensure fair comparison when benchmarking against prior work, we report results on the intersection between Defects4J 3.0 and earlier versions, resulting in 830 defects. HumanEval-Java consists of 163 programming tasks designed to evaluate code generation and repair ability, and has been adopted in recent LLM-based APR studies.

4.2 RQ1: Overall Effectiveness

Table 1 compares CIALLO with recent state-of-the-art APR frameworks on Defects4J and HumanEval-Java. To ensure a fair comparison, we report results obtained using GPT-4o, which is also adopted by several recent LLM-based APR systems. A *plausible patch* is defined as a patch that passes all validation tests, while a *correct patch* is a plausible patch manually verified to be semantically correct. Entries marked as “–” indicate that the corresponding results are not reported in the original papers or the dataset is not evaluated.

Overall results. CIALLO achieves state-of-the-art performance on both benchmarks, significantly outperforming all competing approaches. On Defects4J, CIALLO produces 249 correct patches, exceeding the previous best result of 180 patches (D4C [36]) by 38.3%. On HumanEval-Java, CIALLO generates 161 correct patches, surpassing the prior best result of 140 patches (AdverIntent-Agent [43]) by 15.0%. These consistent improvements across both real-world and synthetic benchmarks demonstrate the effectiveness and generality of the proposed framework.

Analysis. The superior performance of CIALLO can be attributed to its structured search-based design. First, the proposed *understanding–repair paradigm* explicitly separates defect analysis from patch generation, enabling more accurate modeling of defect semantics and reducing error propagation during repair. Second, the *Direction Generation* phase transforms repair into a multi-branch exploration process, which significantly improves the diversity and coverage of the search space.

Third, the *Patch Sieve* mechanism performs dominance-based pruning by eliminating candidates that do not reduce the failure set, thereby improving search efficiency and avoiding redundant exploration. Finally, the *Patch Multiplication* phase further expands promising regions of the search space, increasing the likelihood of discovering correct patches beyond the initial plausible solutions. Together, these components enable CIALLO to explore the patch space more effectively than conventional conversational repair approaches.

Fig. 4 further illustrates the repair coverage of different frameworks. The Venn diagram shows that CIALLO not only covers most defects repaired by prior methods, but also fixes a substantial number of defects that are not addressed by any competing framework. This indicates that CIALLO extends repair capability to previously unresolved defect regions, rather than merely improving performance on already solvable cases.

Answer to RQ1: CIALLO achieves state-of-the-art performance on both Defects4J and HumanEval-Java, with improvements of 38.3% and 15.0%, respectively. The results confirm that the proposed search-based repair framework enables more effective and comprehensive exploration of the patch space.

Table 1. Comparison with state-of-the-art APR frameworks on Defects4J and HumanEval-Java.

Approach	Defects4J (830 bugs)		HumanEval-Java (163 bugs)	
	Plausible	Correct	Plausible	Correct
SelfAPR[41]	130	110	-	-
AlphaRepair[33]	109	79	-	-
ITER[42]	119	74	-	-
ChatRepair[35]	-	162	137	130
Cigar[8]	185	69	152	102
ContrastRepair[16]	201	143	151	137
RepairAgent[2]	186	164	-	-
AdverIntent-Agent[43]	180	141	155	140
D4C[36]	210	180	-	-
Ciallo (GPT-4o)	439	249	161	161

4.3 RQ2: Robustness Across LLMs

To evaluate whether the effectiveness of CIALLO is independent of the underlying large language model (LLM), we conduct experiments using two representative models: *GPT-4o* and *DeepSeek-V3-0324*. These models differ substantially in architecture and training data, providing a strong basis for assessing the robustness and generality of the proposed framework.

To isolate the contribution of CIALLO, we adopt a controlled baseline (*BaseLLM*), which uses the same repair inputs and iteration budget but directly generates patches without structured reasoning. Let n_B and n_F denote the number of plausible/correct patches produced by the baseline and CIALLO, respectively. We measure the relative improvement as:

$$I_R = \frac{n_F - n_B}{n_B} \times 100\% \quad (5)$$

Table 2 presents the results of CIALLO employing GPT-4o and Deepseek-V3-0324 separately.

Table 2. Repair effectiveness of CIALLO under different LLMs (GPT-4o and DeepSeek-V3), compared with corresponding baseline models.

	Plausible	Correct		Plausible	Correct
<i>Defects4J 3.0[14] (854 bugs)</i>					
BaseGPT-4o	302	177	BaseDeepseek-V3	324	195
Ciallo(GPT-4o)	454	253	Ciallo(Deepseek-V3)	480	286
Improvement	50.3%	42.9%		48.2%	46.7%
<i>HumanEval-Java[13] (163 bugs)</i>					
BaseGPT-4o	159	151	BaseDeepseek-V3	157	151
Ciallo(GPT-4o)	161	161	Ciallo(Deepseek-V3)	162	160
Improvement	1.3%	6.6%		3.2%	6.0%

Overall performance. As shown in Table 3, CIALLO achieves strong and competitive performance across all defect types, demonstrating its ability to generalize to diverse repair scenarios. In particular, it achieves the best results on single-line defects, indicating that the framework can effectively resolve localized errors that require limited contextual reasoning.

Performance on complex defects. Beyond simple cases, CIALLO maintains strong performance on more complex defect types, including single-hunk and single-function defects, suggesting that the framework can effectively utilize broader contextual information. Notably, CIALLO achieves competitive or state-of-the-art performance on beyond-function defects, which require reasoning over global program context (e.g., incorrect global variables or missing implementations). These defects are particularly challenging because they cannot be resolved through local edits alone and require coherent reasoning across multiple program components.

Analysis. The effectiveness of CIALLO across different defect types can be attributed to its structured search-based repair process. For simple defects, the framework benefits from accurate defect understanding and localized patch generation, which quickly converges to correct solutions. For more complex defects, the Direction Generation phase enables exploration of multiple repair hypotheses, while the Patch Sieve mechanism iteratively prunes ineffective candidates, allowing the search process to focus on semantically consistent solutions. This combination of exploration and pruning enables CIALLO to adapt to varying levels of defect complexity.

Discussion. We observe that precise defect localization can further improve repair performance, especially for simple defect types. However, CIALLO does not rely on such information to achieve strong results, as demonstrated by its performance on beyond-function defects. This indicates that the proposed framework is capable of reasoning under incomplete or coarse-grained inputs, which is essential for real-world repair scenarios.

Answer to RQ3: CIALLO achieves strong performance across all defect types, with the best results on single-line defects and competitive performance on complex beyond-function defects. These results demonstrate that the proposed search-based framework effectively adapts to both localized and global repair scenarios, enabling robust performance across defects of varying complexity.

Table 3. Repair performance across defect types on Defects4J, comparing CIALLO with representative state-of-the-art APR frameworks.

Approach	SL	SH	SF	BF
ChatRepair[35]	111	26	25	0
ContrastRepair[16]	56	69	75	0
SelfAPR[41]	74	13	11	12
RepairAgent[2]	93	29	22	20
Ciallo(GPT-4o)	111	60	59	19
Ciallo(Deepseek-V3)	120	66	73	21
Defects4J 2.0 Total	157	114	202	368

4.5 RQ4: Ablation Study

To evaluate the contribution of each component in CIALLO, we conduct an ablation study by systematically removing key modules from the structured repair pipeline, including **understanding** (defect diagnosis), **direction** (repair planning), and **patch sieve** (search pruning). DeepSeek-V3 is used as the base LLM. Experiments are performed on Defects4J 3.0 and HumanEval-Java, with results shown in Table 4.

Overall impact. Removing any component leads to a degradation in repair performance, confirming that all modules are necessary for the effectiveness of the framework. This result validates that CIALLO relies on a coordinated search process, rather than isolated improvements from individual components.

Effect of direction module. Removing the **direction** module results in the most significant performance drop. This demonstrates that explicit decomposition of the repair task into multiple search directions is critical for effective exploration of the patch space. Without this component, the repair process degenerates into unguided trial-and-error, leading to inefficient and incomplete exploration.

Effect of understanding and patch sieve. A notable phenomenon is observed when removing either the **understanding** or **patch sieve** module: the number of *plausible* patches increases, while the number of *correct* patches decreases. This indicates that the search space becomes larger but significantly less precise. Without proper defect understanding, the search process is guided by incorrect hypotheses; without pruning, the exploration easily diverges into unproductive regions. As a result, more patches satisfy the test suite but fail to preserve correct semantics, highlighting the importance of precision control in LLM-based repair.

Mechanism analysis. These observations reveal a clear division of roles within the CIALLO framework. The **direction** module governs exploration by expanding the search space through multiple hypotheses, while the **understanding** and **patch sieve** modules enforce *precision* by ensuring that exploration is guided by correct reasoning and constrained through pruning. This balance between exploration and precision is essential for effective program repair, particularly in complex defect scenarios.

Answer to RQ4: All components in CIALLO are necessary and jointly form a structured search process. The direction module enables effective exploration of the patch space, while the understanding and patch sieve modules ensure precision by guiding and pruning

the search. The ablation results demonstrate that removing any component disrupts this balance, leading to inefficient exploration or reduced correctness.

Table 4. Ablation Experiments.

Approach	<i>Defects4J 3.0[14] (854 bugs)</i>		<i>HumanEval-Java[13] (163 bugs)</i>	
	Plausible	Correct	Plausible	Correct
BaseDeepseek-V3	324	195	157	151
w/o direction	436	245	162	156
w/o understanding	486	268	162	160
w/o patch sieve	485	271	163	160
Ciallo(Deepseek-V3)	480	286	162	160

4.6 RQ5: Efficiency Analysis

To evaluate the efficiency of CIALLO from a search perspective, we analyze its token consumption and monetary cost, which are critical for the practical deployment of LLM-based program repair systems. Following prior work [43], we compare CIALLO with state-of-the-art frameworks using publicly reported cost data. Fig. 5(a) shows the distribution of token usage, while Fig. 5(b) presents the corresponding monetary cost based on median consumption.

Overall efficiency. CIALLO consistently achieves lower token consumption than competing frameworks in the majority of cases. Although higher token usage may occur in a small number of difficult instances (e.g., when handling complex defects with large contexts), the overall distribution remains significantly more efficient.

Median cost and stability. The median cost of CIALLO is particularly low, with an average of approximately 25K tokens per defect, corresponding to about 0.76 cents under GPT-4o pricing. This is nearly an order of magnitude lower than existing approaches, indicating strong efficiency in typical repair scenarios.

Trade-off analysis. Importantly, the efficiency gain does not come at the expense of effectiveness. As demonstrated in RQ1, CIALLO achieves state-of-the-art repair performance while maintaining low resource consumption. This indicates that the proposed framework improves both efficiency and effectiveness simultaneously, rather than trading one for the other.

Mechanism analysis. The efficiency of CIALLO can be attributed to its structured search-based design. The Direction Generation phase reduces redundant exploration by guiding the search toward diverse yet meaningful hypotheses, while the Patch Sieve mechanism prunes unproductive candidates early, preventing unnecessary expansion of the search space. Together, these components enable efficient exploration and refinement, leading to lower token consumption.

Answer to RQ5: CIALLO achieves substantially lower repair cost than existing frameworks, with a median usage of approximately 25K tokens (about 0.76 cents per defect). This efficiency is achieved through structured exploration and pruning, allowing the framework to reduce redundant search while maintaining state-of-the-art effectiveness.

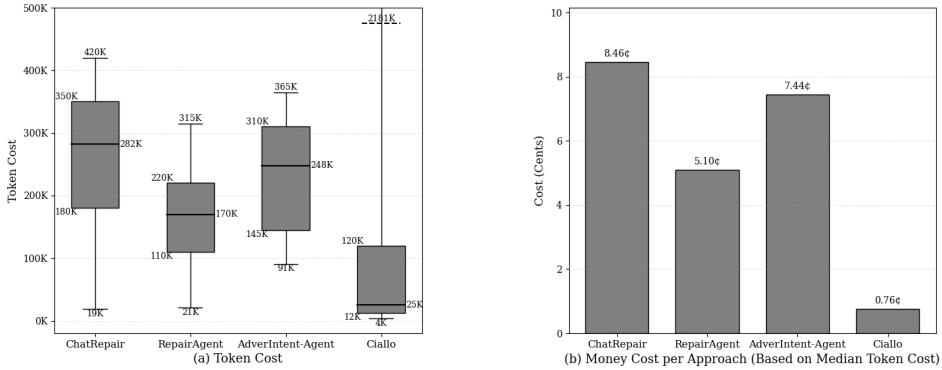


Fig. 5. Cost-efficiency comparison on Defects4J. (a) Distribution of token consumption per defect (box plot). (b) Estimated monetary cost based on median token usage. CIALLO achieves significantly lower median cost than existing frameworks while maintaining strong repair performance.

4.7 RQ6: Search Dynamics Analysis

To better understand how CIALLO explores and refines the patch space, we analyze its search dynamics during repair. Specifically, we examine (1) the number of generated patches, (2) the impact of the Patch Sieve mechanism on pruning, and (3) the distribution of LLM calls. These analyses provide insights into how structured exploration and pruning contribute to effective program repair.

Effect of Patch Sieve on search space. Fig. 6 compares the number of patches generated with and without the Patch Sieve mechanism. We observe that removing Patch Sieve leads to a substantial increase in the number of generated patches. In particular, the maximum number of patches per defect increases from 17 to 59, indicating a significantly larger and less controlled search space.

Pruning effectiveness. Table 5 further quantifies this effect. Without Patch Sieve, the total number of plausible patches increases from 1714 to 2238, while the number of fixed defects decreases from 286 to 271. This result clearly shows that blindly expanding the search space does not improve repair performance. Instead, Patch Sieve effectively prevents the LLM from venturing down search paths involving unproductive candidates, leading to a smaller yet more informative search space that improves correctness.

LLM call distribution. Fig. 7 presents the distribution of LLM calls. We observe that the majority of calls are concentrated in the initial patch generation stage (68.9%), followed by defect understanding (23.6%) and patch multiplication (7.6%). This indicates that CIALLO focuses most of its computational budget on exploring candidate patches, while keeping refinement costs relatively low. Furthermore, the number of LLM calls per defect is generally moderate (median around 17 calls), although a small number of complex defects require deeper search (up to 532 calls), suggesting adaptive allocation of computational resources.

Analysis. These results reveal that CIALLO follows a structured search process that balances exploration and pruning. The Direction Generation phase expands the search space through diverse hypotheses, while the Patch Sieve mechanism constrains the search by eliminating candidates that do not reduce the failure set. This prevents the search process from diverging into unproductive regions. At the same time, the distribution of LLM calls shows that the framework allocates effort adaptively based on defect complexity, focusing more resources on challenging cases.

Answer to RQ6: CIALLO explores the patch space through a structured and adaptive search process. The Patch Sieve mechanism effectively reduces redundant candidates, resulting in a smaller yet more productive search space. Combined with adaptive allocation of computational effort, this design enables efficient and scalable program repair.

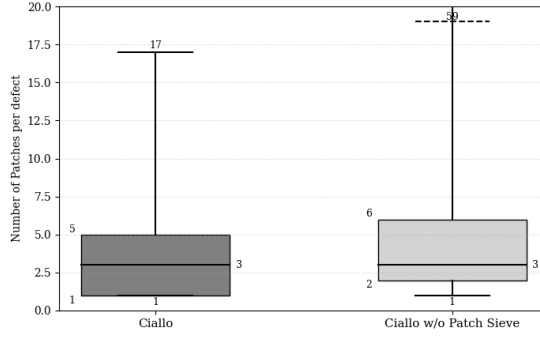


Fig. 6. Comparison of the number of patches generated by CIALLO with and without Patch Sieve

Table 5. Total number of generated patches.

Approach	Ciallo	w/o Patch Sieve
Total Plausible Patches	1714	2238
Fixed Defects	286	271

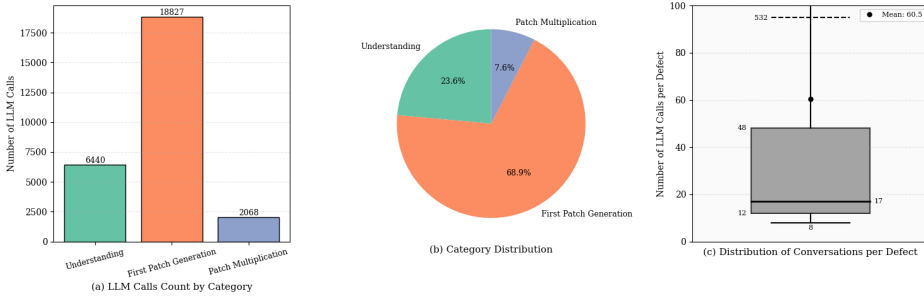


Fig. 7. LLM calls distribution

5 THREATS TO VALIDITY

We discuss potential threats to validity from both internal and external perspectives.

Internal threats:

- **Data leakage.** CIALLO employs GPT-4o and DeepSeek-V3, which are released after the Defects4J benchmark, introducing potential risk of data leakage. To mitigate this concern, we follow prior work [16, 43] and additionally evaluate on HumanEval-Java, which was released after GPT-4o. Moreover, as shown in Section 4.3, CIALLO achieves consistent improvements across different LLMs. This cross-model consistency suggests that the observed performance gains are not attributable to memorization or data leakage, but stem from the proposed structured repair framework.
- **Language generality.** Our evaluation is conducted on Java benchmarks (Defects4J and HumanEval-Java), which may limit generalizability to other programming languages. However, CIALLO operates on semantic-level representations and structured search mechanisms, which are largely language-agnostic. Given the strong cross-language capabilities of modern LLMs, we expect the framework to generalize to other mainstream languages, which we leave for future validation.
- **Hyperparameter settings.** The choice of hyperparameters (e.g., repair iterations and context length) follows prior studies [16, 34, 35]. While different configurations may affect absolute performance, all methods are evaluated under the same settings, ensuring fair comparison. Therefore, this threat is unlikely to affect the relative conclusions. Future work may further explore adaptive or data-driven parameter tuning strategies.

External threats:

- **Manual patch validation.** Determining semantic correctness of patches remains a challenge in APR. We adopt a two-stage validation process: (1) structural equivalence checking using abstract syntax trees against ground-truth patches, and (2) manual validation for remaining plausible patches. Two independent reviewers perform cross-validation, and only patches agreed upon by both are considered correct. Detailed validation criteria are provided in Appendix A. All experimental data are publicly available to support reproducibility.¹

6 CONCLUSION

In this paper, we presented CIALLO, a structured search-based framework for automated program repair using large language models. We introduced the *understanding–repair paradigm*, which explicitly separates defect understanding from repair execution, and formulated program repair as a controlled search process over the patch space. Building on this perspective, CIALLO integrates semantic grounding, hypothesis generation, and iterative pruning into a unified framework. The direction-guided search strategy enables systematic exploration of diverse repair hypotheses, while the Patch Sieve mechanism effectively eliminates unproductive candidates, resulting in both improved repair effectiveness and reduced search cost. Extensive experiments on Defects4J and HumanEval-Java demonstrate that CIALLO achieves state-of-the-art performance, while significantly improving search efficiency. Further analyses show that the proposed framework generalizes across different models and defect types, and adapts its search behavior according to defect complexity. More broadly, this work suggests that automated program repair with LLMs should be viewed as a structured search problem rather than a one-shot generation task. By explicitly modeling exploration, pruning, and refinement, LLM-based repair systems can achieve more reliable, efficient, and controllable behavior.

In future work, we plan to extend CIALLO along several directions. First, we will generalize the framework to additional programming languages and larger codebases, to further validate its applicability in diverse real-world scenarios. Second, we aim to extend the framework to more complex software engineering tasks, such as code synthesis, bug localization, and automated testing, where

¹All our experimental data are available at *our GitHub repository*

structured search and reasoning are also critical. Finally, we will investigate adaptive strategies for balancing exploration and precision in the repair process, including dynamic adjustment of search depth, pruning criteria, and resource allocation. We believe that such adaptive mechanisms will further improve the scalability and robustness of search-based LLM-driven software engineering systems.

Acknowledgments

The authors would like to thank the open-source community for providing valuable tools and datasets that made this research possible. We also thank the anonymous reviewers for their constructive comments and helpful suggestions. This work was supported by the Guangdong Natural Science Foundation under Grant No. 2025A1515011397.

References

- [1] Avinash Anand, Akshit Gupta, Nishchay Yadav, and Shaurya Bajaj. 2024. A comprehensive survey of ai-driven advancements and techniques in automated program repair and code generation. *arXiv preprint arXiv:2411.07586* (2024).
- [2] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 694–694.
- [3] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1943–1959.
- [4] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [5] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and Accurate Source Code Differencing. In *Proceedings of the International Conference on Automated Software Engineering*. 313–324. doi:10.1145/2642937.2642982
- [6] Sichong Hao, Xianjun Shi, and Hongwei Liu. 2024. Exploring the potential of pre-trained language models of code for automated program repair. *Electronics* 13, 7 (2024), 1200.
- [7] Sichong Hao, Xianjun Shi, Hongwei Liu, and Yanjun Shu. 2023. Enhancing code language models for program repair by curricular fine-tuning framework. In *2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 136–146.
- [8] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. 2024. Cigar: Cost-efficient program repair with llms. *arXiv preprint arXiv:2402.06598* (2024).
- [9] Dong Huang, Qingwen Bu, and Heming Cui. 2023. Codcot and beyond: Learning to program and test like a developer. *arXiv preprint arXiv:2308.08784* 23 (2023).
- [10] Zhili Huang, Ling Xu, Chao Liu, Weifeng Sun, Xu Zhang, Yan Lei, Meng Yan, and Hongyu Zhang. 2025. DynaFix: Iterative Automated Program Repair Driven by Execution-Level Dynamic Information. *arXiv preprint arXiv:2512.24635* (2025).
- [11] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [12] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1430–1442.
- [13] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 1430–1442. doi:10.1109/ICSE48619.2023.00125
- [14] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440.
- [15] Dongsun Kim, Jaechang Nam, Jaewoo Song, and Sunghun Kim. 2013. Automatic patch generation learned from human-written patches. In *2013 35th international conference on software engineering (ICSE)*. IEEE, 802–811.
- [16] Jialong Kong, Xiaofei Xie, Mingfei Cheng, Shangqing Liu, Xiaoning Du, and Qi Guo. 2025. Contrastrepair: Enhancing conversation-based automated program repair via contrastive test case pairs. *ACM Transactions on Software Engineering and Methodology* 34, 8 (2025), 1–31.
- [17] Hung Le, Hailin Chen, Amrita Saha, Akash Gokul, Doyen Sahoo, and Shafiq Joty. 2023. Codechain: Towards modular code generation through chain of self-revisions with representative sub-modules. *arXiv preprint arXiv:2310.08992*

- (2023).
- [18] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2011. Genprog: A generic method for automatic software repair. *Ieee transactions on software engineering* 38, 1 (2011), 54–72.
 - [19] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65.
 - [20] Yi Li, Shaohua Wang, and Tien N Nguyen. 2020. Dlfix: Context-based code transformation learning for automated program repair. In *Proceedings of the ACM/IEEE 42nd international conference on software engineering*. 602–614.
 - [21] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large language model-based agents for software engineering: A survey. *ACM Transactions on Software Engineering and Methodology* (2024).
 - [22] Ehsan Mashhadi and Hadi Hemmati. 2021. Applying codebert for automated program repair of java simple bugs. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 505–509.
 - [23] Martin Monperrus. 2018. Automatic software repair: A bibliography. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–24.
 - [24] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. Semfix: Program repair via semantic analysis. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 772–781.
 - [25] Francisco Ribeiro, Rui Abreu, and João Saraiva. 2022. Framing program repair as code completion. In *Proceedings of the Third International Workshop on Automated Program Repair*. 38–45.
 - [26] Douglas C Schmidt, Jesse Spencer-Smith, Quchen Fu, and Jules White. 2024. Towards a catalog of prompt patterns to enhance the discipline of prompt engineering. *ACM SIGAda Ada Letters* 43, 2 (2024), 43–51.
 - [27] Weishi Wang, Yue Wang, Shafiq Joty, and Steven CH Hoi. 2023. Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 146–158.
 - [28] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
 - [29] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382* (2023).
 - [30] Tongshuang Wu, Ellen Jiang, Aaron Donsbach, Jeff Gray, Alejandra Molina, Michael Terry, and Carrie J Cai. 2022. Promptchainer: Chaining large language model prompts through visual programming. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. 1–10.
 - [31] Chunqiu Steven Xia, Yifeng Ding, and Lingming Zhang. 2023. The plastic surgery hypothesis in the era of large language models. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 522–534.
 - [32] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.
 - [33] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: revisiting automated program repair via zero-shot learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971.
 - [34] Chunqiu Steven Xia and Lingming Zhang. 2023. Keep the Conversation Going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. *arXiv preprint arXiv:2304.00385* (2023).
 - [35] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 819–831.
 - [36] Junjielong Xu, Ying Fu, Shin Hwei Tan, and Pinjia He. 2025. Aligning the Objective of LLM-Based Program Repair. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2548–2560.
 - [37] Boyang Yang, Zijian Cai, Fengling Liu, Bach Le, Lingming Zhang, Tegawendé F Bissyandé, Yang Liu, and Haoye Tian. 2025. A survey of LLM-based automated program repair: Taxonomies, design paradigms, and applications. *arXiv preprint arXiv:2506.23749* (2025).
 - [38] Boyang Yang, Luyao Ren, Xin Yin, Jiadong Ren, Haoye Tian, and Shunfu Jin. 2025. Input reduction enhanced llm-based program repair. *arXiv preprint arXiv:2507.15251* (2025).
 - [39] Yaopeng Yang, Chuanqi Li, Zhifeng Han, Rui Li, Kui Xu, Qingyuan Li, Wenkang Zhong, Zongwen Shen, Zhiwei Fei, Jidong Ge, et al. 2025. Patch Generation in APR: A Survey from the Perspectives of Utilizing LLMs and Using APR-Specific Information. *ACM Transactions on Software Engineering and Methodology* (2025).
 - [40] Yao Yao, Zuchao Li, and Hai Zhao. 2023. Beyond chain-of-thought, effective graph-of-thought reasoning in language models. *arXiv preprint arXiv:2305.16582* (2023).

Example 1. Semantic Identity

```
public static double triangle_area(double a, double h) {
    return a * h / 2.;
    return (a * h) / 2;
}
```

- [41] He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [42] He Ye and Martin Monperrus. 2024. Iter: Iterative neural repair for multi-location patches. In *Proceedings of the 46th IEEE/ACM international conference on software engineering*. 1–13.
- [43] He Ye, Aidan ZH Yang, Chang Hu, Yanlin Wang, Tao Zhang, and Claire Le Goues. 2025. Adversarial Reasoning for Repair Based on Inferred Program Intent. *arXiv preprint arXiv:2505.13008* (2025).
- [44] Wei Yuan, Qunjun Zhang, Tieke He, Chunrong Fang, Nguyen Quoc Viet Hung, Xiaodong Hao, and Hongzhi Yin. 2022. CIRCLE: Continual repair across programming languages. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 678–690.
- [45] Qihao Zhu, Zeyu Sun, Yuan-an Xiao, Wenjie Zhang, Kang Yuan, Yingfei Xiong, and Lu Zhang. 2021. A syntax-guided edit decoder for neural program repair. In *Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 341–353.

A Standards for Patch Validation

Accurately determining the semantic correctness of patches remains a fundamental challenge in automated program repair (APR). To ensure consistent and reproducible evaluation, we establish a structured validation framework that categorizes patch equivalence into several representative cases. This framework provides systematic guidance for assessing semantic correctness beyond test-based validation, and can serve as a reference for future APR studies.

A.1 AST Equivalence

We first consider structural equivalence, where the patch under validation is identical to the reference patch at the abstract syntax tree (AST) level. AST equivalence implies functional identity, allowing differences only in formatting (e.g., whitespace or comments). In practice, AST differences can be automatically detected using tools such as Gmtree Spoon AST Diff [5]. Patches that are AST-equivalent are directly considered correct, while patches with AST differences require further analysis.

A.2 Identifier-Level Variations

In some cases, patches differ only in identifier naming (e.g., local variable names), which does not affect program semantics. Such variations are treated as semantically equivalent, even though AST comparison tools may report differences.

A.3 Semantic Identity

This category includes patches that are mathematically or logically equivalent, despite syntactic differences. Typical examples include algebraic transformations, equivalent logical expressions, or alternative implementations such as $i++$ versus $i += 1$. Special care is required for cases involving numerical operations (e.g., integer division truncation), where subtle semantic differences may arise. As shown in Example 1 and Example 2, the red part represents the implementation of the standard correct patch, while the green part represents the implementation of the patch require validation.

Example 2. Semantic Identity

```
public static int starts_one_ends(int n) {
    if (n == 1)
        return 1;
    return (int) ((10 + 9 - 1) * Math.pow(10, n - 2));
    return 18 * (int) Math.pow(10, n - 2);
}
```

Example 3. Control-Flow Equivalence

```
public int compare(Integer o1, Integer o2) {
    int num_ones_o1 = number_ones(o1);
    int num_ones_o2 = number_ones(o2);
    if(num_ones_o1 == num_ones_o2) {
        if(o1 == o2) return 0;
        if(o1 < o2) return -1;
        if(o1 > o2) return 1;
        if(o1 < o2) return -1;
        if(o1 > o2) return 1;
        return 0;
    }
    if(num_ones_o1 < num_ones_o2) return -1;
    else return 1;
}
```

A.4 Equivalent Control Flow Statements

Patches may differ in control flow structure while preserving equivalent execution semantics. Examples include reordering conditional branches (as shown in Example 3) or transformations between loop constructs (e.g., *for* and *while*). Such patches are considered correct if their execution behavior is equivalent across all relevant inputs.

Additional caution is required when new conditions are introduced (e.g., null checks), to ensure consistency with the intended behavior.

A.5 Equivalent Implementation

Some patches implement the same functionality using different coding strategies, such as restructuring loops, introducing auxiliary variables, or modifying algorithmic structure. Examples 4 and 5 provide two specific illustrations. Validating such cases requires reasoning about program semantics and ensuring that the patch fully satisfies the intended behavior.

Boundary conditions (e.g., null inputs or edge cases) should be explicitly considered during validation.

A.6 Inline and Method-Based Implementation

Patches may inline external method logic or replace inline logic with method calls as shown in Example 6. These variations are considered equivalent if the invoked or inlined logic is semantically consistent with the reference implementation. However, if semantic equivalence cannot be confidently established due to complex call chains, the patch is conservatively treated as incorrect.

Example 4. Equivalent Implementation

```
String encode_cyclic(String s) {
    ArrayList<String> groups = new ArrayList<String>();

    for (int i = 0; i < s.length() - s.length() % 3; i += 3){
        for (int i = 0; i < s.length(); i += 3){
            groups.add(s.substring(i, Math.min(s.length(), i + 3)));
        }

        String result = "";
        for (String group : groups){
            result += group.substring(1) + group.substring(0, 1);
            if(group.length() == 3) {
                result += group.substring(1) + group.substring(0, 1);
            } else {
                result += group;
            }
        }

        return result;
    }
}
```

Example 5. Equivalent Implementation

```
public static List<Integer> unique(List<Integer> l) {
    List<Integer> result = new ArrayList<Integer>();
    for (Integer n : l){
        if (result.contains(n))
            continue;
        result.add(n);
    }
    Set<Integer> uniqueSet = new HashSet<Integer>(l);
    List<Integer> result = new ArrayList<Integer>(uniqueSet);
    Collections.sort(result);
    return result;
}
```

Example 6. Inline vs Method-Based Implementation

```
public Iterator<Chromosome> iterator() {
    return getChromosomes().iterator();
    return Collections.unmodifiableList(chromosomes).iterator();
}
public List<Chromosome> getChromosomes() {
    return Collections.unmodifiableList(chromosomes);
}
```

A.7 Insignificant Differences

Certain differences are considered semantically irrelevant, such as variations in exception messages or minor numerical discrepancies caused by implementation choices. As shown in the bisection method algorithm for finding real roots of a polynomial in Example 7, although different initial value selections may lead to variations in the final result, the algorithm will eventually converge to

Example 7. Insignificant Differences

```
public static double find_zero(List<Double> xs){
    double begin = -1.;
    double end = 1.;
    double begin = -10.;
    double end = 10.;

    while (poly(xs, begin) * poly(xs, end) > 0) {
        begin *= 2.;
        end *= 2.;
    }
    while (end - begin > 1e-10) {
        double center = (begin + end) / 2.;

        if (poly(xs, center) * poly(xs, begin) > 0){
            begin = center;
        } else {
            end = center;
            if (poly(xs, begin) * poly(xs, center) < 0) {
                end = center;
            } else {
                begin = center;
            }
        }
    }
    return begin;
}
```

the desired precision range without affecting the correctness of the program. Unless such differences directly relate to the defect, they are ignored during validation.

A.8 Conservative Validation Principle

In cases where semantic equivalence cannot be confidently determined, patches are conservatively classified as incorrect. As a result, the number of correct patches reported in this study should be interpreted as a conservative estimate.

B Details of Execute Patch Generation Function

This section presents the detailed procedure of the patch generation process in CIALLO. The function `EXECUTEPATCHGENERATION` is responsible for iteratively generating, validating, and refining candidate patches under different repair modes. It supports both the initial patch generation phase and the patch multiplication phase through a unified framework.

Algorithm 2 Patch generation procedure in CIALLO

Input: Defective program δ , generation strategy μ_i , set of direction Γ , test suite τ , maximum context length $\lambda_{\max}$, generation state φ

Output: Set of generated patches Π_i

```

1: function EXECUTEPATCHGENERATION( $\delta, \mu_i, \Gamma, \tau, \lambda_{\max}, \varphi$ )
2:   Initialize  $\Pi_h, \Pi_u, \Pi_c, \Pi_i \leftarrow \emptyset$ 
3:    $\lambda_{\text{current}} \leftarrow 0, \kappa_{\text{prev}} \leftarrow 0$ 
4:   while  $\lambda_{\text{current}} < \lambda_{\max} \wedge \neg \text{PhaseStopConditionMet}$  do
5:      $\lambda_{\text{current}} \leftarrow \lambda_{\text{current}} + 1$ 
6:      $\Pi_g \leftarrow \emptyset$ 
7:     if  $\varphi = \text{FirstPlausiblePatchGeneration}$  then
8:       if  $\lambda_{\text{current}} = 1$  then
9:         for each  $\gamma_i \in \Gamma$  do
10:           $\pi_j \leftarrow \text{CallLLM}(\text{BuildRepairPrompt}(\delta, \mu_i, \gamma_i, \tau))$ 
11:           $\Pi_g.\text{add}(\pi_j)$ 
12:       else
13:          $(\Psi, \Pi_h) \leftarrow \text{BuildOngoingRepairPrompt}(\delta, \mu_i, \Pi_c, \Pi_h)$ 
14:         for each  $\psi_i \in \Psi$  do
15:           $\pi_j \leftarrow \text{CallLLM}(\psi_i)$ 
16:           $\Pi_g.\text{add}(\pi_j)$ 
17:       else if  $\varphi = \text{PatchMultiplication} \wedge \Pi_i \neq \emptyset$  then
18:          $\Pi_p \leftarrow \text{GetPlausiblePatches}(\Pi_i)$ 
19:         if  $\text{PlausiblePatchesCount}(\Pi_i) = \kappa_{\text{prev}}$  then
20:            $\text{ReconstructUsedPlausiblePatches}(\Pi_u, \tau_n, \Pi_p)$ 
21:          $\kappa_{\text{prev}} \leftarrow \text{PlausiblePatchesCount}(\Pi_i)$ 
22:         for each  $\pi_f \in (\Pi_p - \Pi_u)$  do
23:           if  $\text{SufficientPlausiblePatches}(\tau_n)$  then
24:             break
25:            $\pi_j \leftarrow \text{CallLLM}(\text{BuildMultiplicationPrompt}(\delta, \mu_i, \pi_f, \Pi_h))$ 
26:            $\Pi_g.\text{add}(\pi_j)$ 
27:         for each  $\pi_j \in \Pi_g$  do
28:            $\iota_t \leftarrow \text{Validate}(\pi_j)$ 
29:           if  $\pi_j$  not processed  $\wedge (\lambda_{\text{current}} = 1 \vee \text{PatchSieve}(\pi_j))$  then
30:              $\Pi_i.\text{add}(\pi_j)$ 
31:              $\Pi_c.\text{add}(\pi_j, \iota_t)$ 
32:   return  $\Pi_i$ 

```

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009